

**PENGEMBANGAN KNOWLEDGE GRAPH-BASED RETRIEVAL AUGMENTED
GENERATION DENGAN HYBRID VECTOR-GRAPH RETRIEVAL UNTUK ASISTEN
PENCARIAN LITERATUR AKADEMIK**

PROPOSAL TUGAS AKHIR



Oleh
RIZKY YANUAR KRISTIANTO
NIM 22031554017

UNIVERSITAS NEGERI SURABAYA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
PROGRAM STUDI SARJANA SAINS DATA
2026

HALAMAN PERSETUJUAN PROPOSAL TUGAS AKHIR BENTUK: SKRIPSI

Nama Mahasiswa : Rizky Yanuar Kristianto
NIM : 22031554017
Judul Penelitian : Pengembangan *Knowledge Graph-Based Retrieval Augmented Generation* dengan *Hybrid Vector-Graph Retrieval* untuk Asisten Pencarian literatur Akademik

ini telah disetujui dan dinyatakan memenuhi syarat untuk diajukan dalam ujian proposal tugas akhir.

Surabaya, 17 juni 2026
Pembimbing I,

Ibnu Febry K., S.Kom., M.Sc., Ph.D.
NIP 198802182015041001

Pembimbing II,

Hasanuddin Al-Habib, M.Si.
NIP 199308092022031010

LEMBAR PENGESAHAN REVISI PROPOSAL TUGAS AKHIR

Nama Mahasiswa : Rizky Yanuar Kristianto
 NIM : 22031554017
 Judul Penelitian : Pengembangan *Knowledge Graph-Based Retrieval Augmented Generation* dengan *Hybrid Vector-Graph Retrieval* untuk Asisten Pencarian literatur Akademik

ini dipertahankan di hadapan dewan penguji pada tanggal 3 Juli 2026 dan telah disetujui serta dinyatakan memenuhi syarat untuk dilanjutkan dalam pengambilan data.

Dewan Penguji,	Tanda Tangan	Tanggal Selesai/Revisi
Ibnu Febry K., S.Kom., M.Sc., Ph.D. NIP 198802182015041001		
Hasanuddin Al-Habib, M.Si. NIP 199308092022031010		
Dr. Atik Wintarti, M.Kom. NIP 196610121991032002		
Yuni Rosita Dewi, M.Si. NIP 199506282024062001		

Mengetahui,
 Koordinator Program Studi
 Sarjana Sains Data

Yuliani Puji Astuti, M.Si.
 NIP. 197807312006042001

DAFTAR ISI

DAFTAR TABEL

DAFTAR GAMBAR

DAFTAR SIMBOL

G	<i>Knowledge Graph</i> akademik yang didefinisikan sebagai himpunan entitas (E), relasi (R), dan <i>triplet</i> (T).
(h, r, t)	<i>Triplet</i> pengetahuan yang terdiri dari <i>head entity</i> (subjek), <i>relation</i> (predikat), dan <i>tail entity</i> (objek).
N_{raw}	Jumlah total data mentah publikasi yang dikumpulkan dari berbagai sumber sebelum pemrosesan.
q	Pertanyaan atau kueri yang diajukan oleh pengguna ke dalam sistem.
$\mathcal{K}_l, \mathcal{K}_h$	Kata kunci hierarkis; tingkat rendah (<i>low-level</i>) untuk pencarian detail dan tingkat tinggi (<i>high-level</i>) untuk konteks tema besar.
$\mathcal{L}_{\text{local}}$	Informasi lokal (mikro) yang mencakup detail spesifik entitas dan relasi dari subgraf.
$\mathcal{L}_{\text{global}}$	Informasi global (makro) yang mencakup pola konseptual dan tren riset lintas domain.
$ V $	Jumlah klaim yang terverifikasi (<i>verified claims</i>) dalam jawaban sistem, digunakan untuk menghitung <i>Faithfulness</i> .
$ S $	Total jumlah klaim (<i>total statements</i>) yang terdapat dalam jawaban yang dihasilkan sistem.
$\text{sim}(q, \hat{q})$	Skor kemiripan (<i>similarity score</i>) antara pertanyaan asli pengguna dengan pertanyaan sintetis untuk mengukur relevansi jawaban.

BAB I

PENDAHULUAN

A. Latar Belakang

Ekosistem penelitian akademik saat ini mengalami ledakan literatur ilmiah yang belum pernah terjadi sebelumnya. Produksi literatur meningkat secara eksponensial, dengan perkiraan jutaan artikel baru diterbitkan setiap tahunnya di berbagai bidang ilmu. Basis data *PubMed* bahkan melaporkan hampir tiga ribu artikel baru terbit setiap hari (?). Sebagai ilustrasi, mesin pencari seperti *Google Scholar* menghasilkan ribuan entri untuk kata kunci “*machine learning*” (?). Meskipun kelimpahan data ini membuka peluang inovasi, volume masif tersebut menimbulkan beban kognitif (*cognitive overload*) bagi peneliti yang harus menavigasi tumpukan literatur untuk menemukan wawasan relevan.

Merespons kondisi tersebut, berbagai mesin pencari literatur seperti *Google Scholar*, *Semantic Scholar*, dan *Microsoft Academic Search* dikembangkan untuk memfasilitasi penemuan publikasi ilmiah (?). Di antara layanan tersebut, *Google Scholar* mendominasi dengan cakupan sekitar 389 juta dokumen (?). Namun, platform ini masih mengandalkan pencarian tradisional berbasis pencocokan kata kunci. ? menyatakan bahwa metode *keyword matching* sering gagal menangkap konteks semantik yang kompleks, sehingga berpotensi melewatkan literatur penting yang menggunakan istilah berbeda namun bermakna serupa. Pendekatan seperti ini menyebabkan sumber daya akademik yang berharga tidak teridentifikasi, yang pada akhirnya dapat menghambat proses transfer pengetahuan dan kolaborasi ilmiah.

Inefisiensi pencarian tersebut semakin krusial di lingkungan institusi pendidikan tinggi seperti UNESA. Berdasarkan data SINTA (?), produktivitas publikasi dosen UNESA menunjukkan tren positif dengan peningkatan rata-rata 20–30% per tahun dalam lima tahun terakhir. Sayangnya, pertumbuhan ini tersebar dalam sistem penyimpanan yang terfragmentasi. Studi oleh ? menyebut fenomena terpisahnya data profil kepakaran dari luaran penelitian dan sitasi sebagai fragmentasi pengetahuan. Akibatnya, pemangku kebijakan menghadapi kesulitan dalam menjawab pertanyaan kompleks yang membutuhkan sintesis dari berbagai sumber data terpisah, bukan sekadar pencarian dokumen tunggal. Metode pencarian konvensional gagal melakukan sintesis ini karena hanya melihat dokumen sebagai entitas terpisah, bukan sebagai jaringan pengetahuan yang saling terkait.

Menjawab tantangan tersebut, *Large Language Models* (LLM) dengan arsitektur *Retrieval-Augmented Generation* (RAG) menawarkan pendekatan baru melalui interaksi bahasa alami terhadap korpus data (?). RAG tradisional, atau *Vector RAG*, memecah dokumen menjadi segmen teks (*chunks*) dan menyimpannya dalam basis data vektor untuk pencarian kemiripan semantik (?). Meskipun efektif untuk kueri sederhana, *Vector RAG* memiliki kelemahan fundamental pada tugas penalaran kompleks. Studi oleh ? menyoroti bahwa

pendekatan vektor memperlakukan informasi sebagai data terisolasi, sehingga gagal menghubungkan titik-titik informasi yang tersebar namun saling terkait di berbagai dokumen.

Kelemahan utama pendekatan vektor murni (*dense retrieval*) terletak pada ketidakmampuannya mengambil informasi lintas sumber dan memahami korpus secara holistik. Dalam konteks akademik, studi oleh ? menekankan bahwa literatur ilmiah saling terhubung melalui sitasi, metodologi, dan kolaborasi penulis. Sistem RAG konvensional seringkali menghasilkan jawaban yang kurang lengkap atau bahkan tidak akurat karena model hanya menebak hubungan informasi yang tidak tertulis jelas dalam teks (?). Risiko ini sangat krusial dalam dunia akademik yang menuntut validitas fakta dan ketertelusuran referensi.

Untuk menjembatani kesenjangan antara kapabilitas LLM dan kebutuhan pengetahuan terstruktur, paradigma *Graph Retrieval-Augmented Generation (GraphRAG)* hadir menyempurnakan *Vector RAG* (?). Berbeda dengan pendekatan vektor, *GraphRAG* mengorganisasikan informasi sebagai *Knowledge Graph (KG)* yang secara eksplisit memetakan entitas (seperti *Dosen, Paper, Topik*) dan relasi (seperti *Menulis, Mensitasi, Membahas*). Survei ? mengindikasikan bahwa *Graph-Based RAG* memungkinkan navigasi relasi, penelusuran sitasi, dan pemahaman konteks hierarkis, yang secara signifikan meningkatkan kemampuan menjawab pertanyaan yang memerlukan *sensemaking* mendalam.

Meskipun menawarkan keunggulan struktural, ketergantungan penuh pada graf saja memiliki tantangan pada kelengkapan informasi yang tidak terstruktur dan biaya komputasi yang tinggi. Oleh karena itu, penelitian ini mengadopsi pendekatan *Knowledge Graph-Based RAG* dengan mekanisme *hybrid retrieval* yang menyinergikan kekuatan pencarian vektor dan graf sebagai solusi jalan tengah. Dalam arsitektur hibrida ini, pencarian vektor berperan menangkap nuansa semantik dan konteks implisit dari teks, sementara struktur graf berfungsi memvalidasi hubungan faktual dan menyediakan kerangka penalaran logis (?). Sinergi ini diharapkan menghasilkan sistem yang tidak hanya “pintar” secara linguistik tetapi juga “akurat” secara faktual.

Berdasarkan permasalahan tersebut, penelitian ini mengusulkan pengembangan sistem asisten pencarian literatur akademik yang mengimplementasikan *Knowledge Graph-Based RAG* dengan mekanisme *Hybrid Retrieval*. Penelitian ini tidak hanya bertujuan meningkatkan metrik akurasi pencarian semata, tetapi juga sejalan dengan tren kecerdasan buatan yang memanfaatkan *Large Language Model* untuk mentransformasi cara sivitas akademika berinteraksi dengan repositori pengetahuan serta mengubah proses tinjauan literatur yang manual menjadi dialog penemuan pengetahuan yang berbasis bukti (*grounded inquiry*).

B. Identifikasi Masalah

1. Pertumbuhan literatur dan rekam jejak publikasi dosen pada domain Informatika dan Komputer di UNESA meningkat serta tersebar pada berbagai sumber, sehingga penelusuran literatur yang komprehensif dan efisien menjadi sulit.
2. Sistem pencarian akademik konvensional mengandalkan pencocokan kata kunci atau kemiripan teks, sehingga kurang mampu memodelkan konteks semantik dan relasi antar entitas akademik yang dibutuhkan untuk penelusuran literatur.

3. Pemanfaatan *Large Language Model* secara mandiri berisiko menghasilkan halusinasi dan jawaban yang tidak dapat ditelusuri sumbernya, sehingga kurang aman untuk kebutuhan akademik yang menuntut akurasi dan keterlacakan.
4. Implementasi *Retrieval-Augmented Generation (RAG)* berbasis *vector similarity* cenderung mengambil potongan teks semata, sehingga kurang optimal untuk pertanyaan yang membutuhkan pemahaman relasional dan pencocokan konteks lintas entitas seperti pencarian literatur.
5. Belum tersedia sistem penelusuran literatur di lingkungan Infokom UNESA yang mengimplementasikan *Knowledge Graph* untuk mendukung generasi jawaban model bahasa besar.

C. Rumusan Masalah

1. Bagaimana merancang *pipeline* konstruksi *Knowledge Graph* yang mampu mengekstraksi dan menstandarisasi entitas serta relasi dari korpus literatur akademik secara otomatis untuk mendukung representasi pengetahuan yang komprehensif?
2. Bagaimana merancang mekanisme *Hybrid Vector-Graph Retrieval* yang mengintegrasikan pencarian vektor dan graf secara sinergis dalam arsitektur *Knowledge Graph Based RAG* pada sistem *Question Answering* untuk meningkatkan relevansi dan kontekstualitas pencarian?
3. Bagaimana efektivitas integrasi *Knowledge Graph* dalam mengurangi halusinasi pada model bahasa besar (*LLM*) dan meningkatkan keterlacakan (*traceability*) referensi dalam jawaban yang dihasilkan?

D. Tujuan Penelitian

1. Merancang *pipeline* konstruksi *Knowledge Graph* yang mampu mengekstraksi dan menstandarisasi entitas serta relasi dari korpus literatur akademik secara otomatis untuk mendukung representasi pengetahuan yang komprehensif.
2. Merancang mekanisme *Hybrid Vector-Graph Retrieval* dalam arsitektur *Knowledge Graph Based RAG* pada sistem *Question Answering* secara sinergis untuk meningkatkan relevansi dan kontekstualitas pencarian.
3. Menganalisis efektivitas integrasi *Knowledge Graph* dalam meminimalkan tingkat halusinasi model bahasa besar (*LLM*) serta meningkatkan keterlacakan (*traceability*) sumber referensi dalam jawaban yang dihasilkan.

E. Manfaat Penelitian

1. Berkontribusi pada kajian *Knowledge Graph* dalam *retrieval-augmented generation* dengan menguji efektivitas pendekatan *hybrid vector-graph retrieval* untuk pencarian literatur akademik institusional.
2. Menyajikan kajian empiris mengenai efektivitas pendekatan hibrida (*Hybrid Vector-Graph*) dalam mengatasi kelemahan metode *RAG* konvensional, khususnya pada domain data berstruktur relasional padat seperti literatur ilmiah.

3. Memperkaya literatur mengenai pemanfaatan *Knowledge Graph* untuk meningkatkan kemampuan penalaran pada *Large Language Models* (LLM).

F. Batasan Masalah

Mengingat luasnya ruang lingkup permasalahan, penelitian ini dibatasi pada aspek-aspek berikut agar pembahasan lebih terfokus:

1. Sumber data yang diproses terbatas pada data tekstual. Elemen non-teks seperti gambar, diagram, dan tabel di dalam dokumen tidak termasuk dalam lingkup pemrosesan.
2. *Input* data utama dibatasi pada metadata publikasi (judul, penulis, tahun, afiliasi) dan teks abstrak, tanpa melakukan pengunduhan isi dokumen PDF.
3. Pengembangan sistem difokuskan pada ekstraksi informasi dari teks tidak terstruktur menjadi *Knowledge Graph*, serta pemanfaatannya sebagai konteks eksternal bagi LLM untuk meminimalkan halusinasi.
4. Pengembangan sistem difokuskan pada penemuan publikasi relevan berbasis topik (*paper discovery*) dan informasi penulis (dosen) beserta bukti pendukung berbasis relasi pada *Knowledge Graph*. Sistem tidak ditujukan sebagai sistem penilaian kinerja dosen atau pemantauan aktivitas akademik secara menyeluruh.
5. Evaluasi sistem dilakukan menggunakan studi kasus data publikasi akademik rumpun Informatika dan Komputer di Universitas Negeri Surabaya (UNESA).

BAB II

KAJIAN PUSTAKA

A. Penelitian Terdahulu

Penelitian ini mengintegrasikan ekstraksi informasi terstruktur dengan metode pencarian hibrida. Rangkuman studi coterdahulu yang melandasi penelitian ini disajikan pada ??.

Tabel 2.1: Tinjauan Penelitian Terdahulu

Judul	Penulis	Kesimpulan
<i>Swamped with Too Many Articles: GraphRAG Makes Getting Started Easy</i>	?	Evaluasi komparatif menunjukkan bahwa <i>GraphRAG</i> mengungguli <i>Naive RAG</i> dan <i>LightRAG</i> dalam empat metrik kritis: <i>Comprehensiveness, Diversity, Empowerment, dan Directness</i> . Temuan ini memvalidasi landasan penelitian bahwa pendekatan berbasis graf menawarkan navigasi semantik yang lebih superior dibandingkan pencarian leksikal untuk mengatasi <i>literature overload</i> pada repositori data berskala besar.
<i>HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction</i>	?	Penelitian ini menetapkan standar efektivitas arsitektur hibrida dengan pencapaian skor <i>Faithfulness</i> dan <i>Answer Relevance</i> sebesar 0,96 serta <i>Context Recall</i> sempurna (1,00) . Hasil empiris ini menjadi rujukan utama penelitian usulan, yang membuktikan bahwa penggabungan <i>Knowledge Graph</i> dan <i>Vector Retrieval</i> adalah strategi optimal untuk memitigasi halusinasi dan memaksimalkan presisi jawaban pada dokumen kompleks.
<i>Knowledge Graph Generation and Application for Unstructured Data Using Data Processing Pipeline</i>	?	Mengembangkan <i>pipeline</i> otomatis menggunakan model generatif <i>REBEL</i> yang mencapai akurasi ekstraksi 87% pada data tidak terstruktur. Metrik akurasi ini menyoroti tantangan yang ada, yang dalam penelitian ini akan ditingkatkan melalui strategi <i>Parallel Processing</i> (integrasi data terstruktur PDDikti/SINTA dan teks abstrak) untuk melampaui batasan akurasi model generatif tunggal dalam konstruksi KG.

B. Kajian Teori

1. *Scholarly Search*

Scholarly search atau pencarian literatur akademik berfokus pada pengelolaan dan pengambilan dokumen ilmiah dari repositori besar, mencakup artikel jurnal, prosiding, tesis, serta metadata terkait (?). Saat ini, domain ini menghadapi tantangan *literature overload*. ? melaporkan bahwa volume publikasi di beberapa bidang berlipat ganda setiap 8 hingga 15 tahun. Dengan lebih dari 200 juta artikel tersedia, peneliti menanggung beban kognitif (*cognitive overload*) yang berat saat mencari referensi secara manual.

Sistem pencarian tradisional seperti *Google Scholar*, *PubMed*, dan *CiteSeerX* memiliki keterbatasan karena mengandalkan sinyal leksikal murni seperti algoritma *BM25* (?). Pendekatan leksikal tersebut seringkali gagal menangkap hubungan makna kompleks antar konsep, kesulitan mengenali sinonim, dan mengabaikan relasi konektivitas antar dokumen. Akibatnya, hasil pencarian sering berupa daftar panjang dokumen yang harus diverifikasi manual oleh peneliti, tanpa indikasi relevansi langsung terhadap pertanyaan riset.

Dokumen ilmiah memiliki karakteristik unik yang membedakannya dari teks biasa. ? menyebutkan bahwa dokumen ilmiah terdiri dari dua dimensi utama: dimensi tekstual (isi dan metadata) serta struktural (jejaring sitasi, ko-sitasi, dan kolaborasi). Keberhasilan pencarian ilmiah di masa depan tidak hanya bergantung pada kemiripan vektor teks (*vector proximity*), tetapi juga pada kemampuan penalaran relasional (*relational reasoning*). Sebagai contoh, ? menunjukkan bahwa pendekatan relasional mampu menghubungkan metodologi dari satu makalah dengan temuan di makalah lain melalui mekanisme *multi-hop reasoning*.

Dalam implementasi praktis, ? menjelaskan bahwa *scholarly search* telah berkembang menjadi beragam tugas spesifik, mulai dari pencarian berbasis topik, penelusuran sitasi, rekomendasi artikel, hingga *question answering* berbasis literatur. Kompleksitas tugas-tugas tersebut menuntut sistem pencarian yang mampu mengintegrasikan berbagai jenis sinyal semantik dan hubungan struktural antar dokumen secara simultan.

Perkembangan terkini, sebagaimana dicatat oleh ?, menunjukkan pemanfaatan *scholarly search* dalam sistem *question answering*. Sistem ini dituntut tidak hanya menampilkan dokumen, tetapi juga menyajikan wawasan lintas bidang, mengidentifikasi celah riset (*research gaps*), dan menyediakan bukti sumber informasi (*traceability*). Urgensi presisi fakta dan navigasi relasional inilah yang mendasari integrasi teknologi *Knowledge Graph* dan *Large Language Models* dalam penelitian ini.

2. *Knowledge Graph*

Knowledge base adalah kumpulan data terorganisir yang mendeskripsikan entitas dan hubungannya dalam format *triplet*. Ketika himpunan *triplet* ini direpresentasikan sebagai jaringan dengan entitas sebagai titik (*node*) dan hubungan sebagai garis (*edge*), strukturnya disebut *Knowledge Graph* (?). Berbeda dengan basis data relasional yang kaku, *Knowledge Graph* menawarkan representasi pengetahuan yang dinamis. Struktur graf ini memungkinkan mesin memahami hubungan antar potongan informasi, menyerupai mekanisme asosiasi kognitif manusia (?).

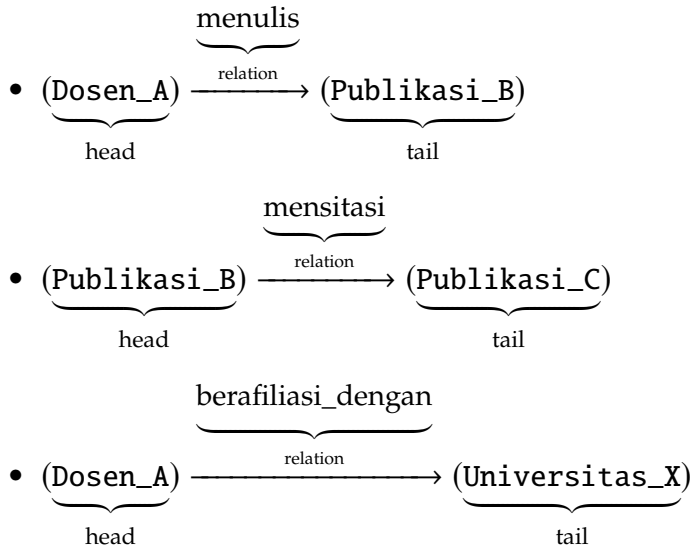
Komponen fundamental dari sebuah *Knowledge Graph* adalah *triplet* faktual yang terdiri dari *Subjek-Predikat-Objek* atau *Head-Relation-Tail*. *Knowledge Graph* menyimpan pengetahuan terstruktur sebagai himpunan *triplet*, yang dapat direpresentasikan sebagai berikut:

$$G = (E, R, T) \quad (2.1)$$

di mana E adalah himpunan entitas (seperti individu atau objek), R adalah himpunan relasi (seperti “bagian_dari”), dan T adalah himpunan *triplet* yang membentuk keseluruhan struktur graf tersebut (?). Setiap elemen di dalam T merepresentasikan satu fakta tunggal yang diformulasikan sebagai:

$$(h, r, t) \in T \quad (2.2)$$

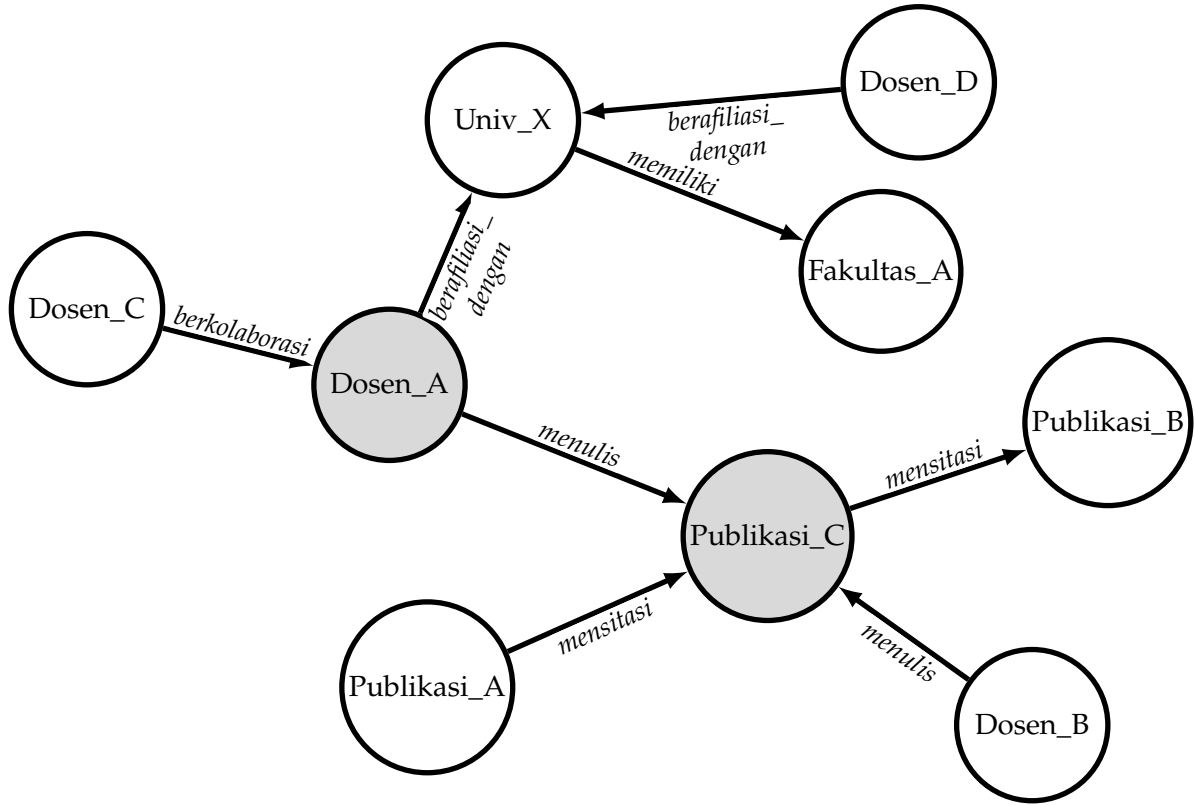
di mana h adalah entitas sumber atau *head*, t adalah entitas tujuan atau *tail*, dan r adalah relasi yang menghubungkan keduanya (?). Contohnya dalam konteks akademik



Struktur ini sangat efektif karena arah panah memberi makna eksplisit, di mana entitas *head* menunjuk langsung ke entitas *tail* melalui garis relasi (*edge*). Jika digambarkan sebagai graf, struktur ini akan terlihat seperti pada ??.

Menurut ?, pengembangan *Knowledge Graph* (KG) terbagi dalam dua paradigma utama: *Resource Description Framework* (RDF) dan *Labeled Property Graph* (LPG). Penelitian ini mengadopsi struktur *LPG* (menggunakan *Neo4j*) karena keunggulannya dalam navigasi hubungan kompleks dan fleksibilitas skema. Keunggulan fundamental KG dibandingkan metode leksikal adalah kemampuan penalaran bertahap (*multi-hop reasoning*), yang memungkinkan sistem menghubungkan entitas yang secara semantik terpisah namun memiliki keterkaitan logis, sebuah fitur vital untuk sintesis akademik.

Dalam lingkungan universitas, penerapan *Scholarly Knowledge Graph* (SKG) memiliki peranan penting dalam mengatasi fenomena *knowledge island* yang dijelaskan oleh ?, yaitu kondisi di mana pengetahuan terisolasi, tidak saling terhubung, dan sulit diintegrasikan satu sama lain. SKG didefinisikan sebagai representasi graf yang memodelkan entitas akademik (seperti penulis, publikasi, dan institusi) beserta hubungan antar mereka secara



Gambar 2.1: Visualisasi *Knowledge Graph* akademik yang menunjukkan relasi antar entitas.

eksplisit. Dalam konteks kecerdasan buatan modern, *Knowledge Graph* (KG) menjadi elemen penting dalam arsitektur *Retrieval-Augmented Generation*, di mana struktur graf digunakan untuk menyediakan konteks faktual kepada *LLM*, mengurangi risiko halusinasi, serta mendukung penalaran multi-langkah yang tidak dapat dicapai oleh metode pencarian konvensional.

3. *Knowledge Extraction*

Knowledge Extraction atau ekstraksi pengetahuan merupakan proses krusial untuk mentransformasi data tidak terstruktur (seperti teks naratif dalam publikasi ilmiah, email, atau dokumen PDF) menjadi representasi pengetahuan yang terorganisir, formal, dan dapat diolah oleh mesin (?). Dalam ekosistem *Knowledge Graph*, *Knowledge Extraction* berperan dalam menyarikan fakta-fakta atomik dari teks untuk mengisi dan memperbarui struktur graf tersebut. Implementasi ini berfungsi sebagai fondasi untuk membangun basis pengetahuan (*knowledge base*) yang dapat memfasilitasi mekanisme penalaran sistem dalam menghasilkan jawaban yang akurat.

Secara formal, tugas *Knowledge Extraction* dapat diformulasikan sebagai fungsi pemetaan dari teks input $\mathcal{T}$ menjadi representasi terstruktur $\mathcal{S}$. Representasi ini terdiri dari himpunan entitas E dan relasi R , sebagaimana dinyatakan dalam Persamaan ??:

$$\mathcal{S} = f_{IE}(\mathcal{T}) = \{(e_i, r_{ij}, e_j) \mid e_i, e_j \in E, r_{ij} \in R\} \quad (2.3)$$

di mana setiap elemen dalam $\mathcal{S}$ berupa *triplet* (e_i, r_{ij}, e_j) yang merepresentasikan satu fakta tunggal, dengan e_i bertindak sebagai entitas subjek, r_{ij} sebagai predikat atau relasi, dan e_j

sebagai entitas objek.

Implementasi fungsi pemetaan berskala besar menuntut otomatisasi, mengingat ekstraksi manual tidak lagi memadai untuk menangani pertumbuhan volume data tekstual yang eksponensial (?). Secara umum, proses otomatisasi *Knowledge Extraction* didekomposisi menjadi tiga tahapan fundamental:

1. *Named Entity Recognition (NER)*

Sebagai langkah awal, *Named Entity Recognition (NER)* bertugas mengidentifikasi dan mengklasifikasikan entitas penting dalam teks untuk menemukan himpunan entitas E (Persamaan ??). NER mendeteksi istilah relevan seperti nama orang, organisasi, atau terminologi ilmiah, lalu memberikan label kategori yang sesuai (?).

2. *Relation Extraction (RE)*

Relation Extraction (RE) bertujuan mengidentifikasi hubungan semantik (r_{ij}) antara dua entitas (e_i dan e_j) yang ditemukan. Proses ini mengklasifikasikan jenis interaksi yang terjadi, misalnya hubungan “menggunakan” antara metode dan algoritma, atau “ditulis oleh” antara publikasi dan peneliti.

3. *Event Extraction (EE)*

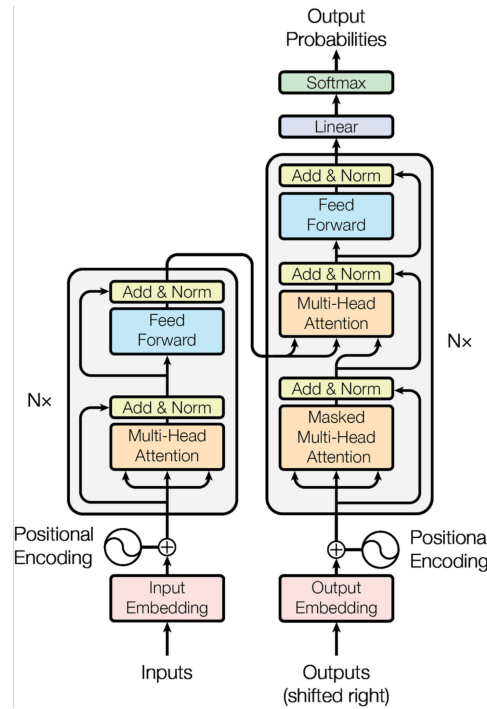
Untuk memperkaya lapisan semantik, *Event Extraction (EE)* kerap ditambahkan. Berbeda dengan RE yang memetakan hubungan statis, EE berfokus pada identifikasi peristiwa dinamis yang mencakup pemicu (*trigger*) serta argumen pendukungnya (pelaku, waktu, lokasi).

Secara metodologis, pendekatan ekstraksi pengetahuan telah berevolusi dari sistem berbasis aturan (*rule-based systems*) dan analisis tata bahasa seperti *dependency parsing* menuju penggunaan model *Deep Learning* yang lebih kompleks, seperti arsitektur BERT dan BiLSTM-CRF (?). Pada paradigma kontemporer, penggunaan Model Bahasa Besar (*Large Language Models*) telah menjadi standar baru melalui teknik *zero-shot* dan *few-shot prompting* untuk melakukan ekstraksi secara otomatis. ? membuktikan bahwa LLM unggul dalam menangkap hubungan semantik yang bersifat implisit dan kaya akan konteks dibandingkan metode tradisional. Meskipun demikian, penggunaan LLM dalam ekstraksi tetap memerlukan mekanisme validasi atau koreksi mandiri (*self-correction*) untuk meminimalkan risiko kesalahan ekstraksi atau halusinasi informasi.

Dalam penelitian ini, efektivitas *Knowledge Extraction* sangat berperan dalam menentukan kualitas pengambilan informasi. Proses ekstraksi yang mendalam terhadap kontribusi riset, seperti metode yang diajukan, dataset yang digunakan, serta metrik hasil, memungkinkan sistem untuk mengaitkan berbagai temuan ilmiah yang tersebar, mempermudah penemuan pola baru, serta memastikan bahwa setiap jawaban yang diberikan dapat ditelusuri kembali ke bukti asli dalam literatur (*traceability*).

4. *Large Language Models (LLM)*

Large Language Models (LLM) merepresentasikan kelas model *Deep Learning* canggih yang dilatih menggunakan sejumlah besar data teks untuk memahami, memproses, dan



Gambar 2.2: Arsitektur Transformer (?)

menghasilkan teks dengan kualitas yang menyerupai bahasa manusia (?). Secara struktural, mayoritas *LLM* modern dibangun di atas arsitektur *Transformer* yang memanfaatkan mekanisme *self-attention* (*multi-head attention*) dan *feed-forward networks*, seperti ditunjukkan pada ?? dan pertama kali diperkenalkan oleh ?. Mekanisme ini memungkinkan model untuk memproses seluruh urutan kata dalam suatu kalimat secara simultan dan menangkap ketergantungan jarak jauh (*long-range dependencies*) antar kata, memberikan pemahaman konteks yang jauh lebih dalam dibandingkan model tradisional seperti *RNN* atau *LSTM* yang memproses data secara sekuensial.

Evolusi *LLM* telah melahirkan dua paradigma utama, yaitu model *encoder-only* seperti *BERT* yang unggul dalam pemahaman kontekstual dan ekstraksi informasi (?), serta model *decoder-only* seperti keluarga *GPT* yang dioptimalkan untuk tugas-tugas generatif *autoregresif* (?). Melalui proses *pre-training* pada triliunan token data dari internet, buku, dan artikel ilmiah, *LLM* menunjukkan fenomena *emergent properties*, di mana kemampuan kompleks seperti penalaran *chain-of-thought*, pemecahan masalah matematika, dan *in-context learning* (ICL) muncul seiring dengan peningkatan skala parameter model (?). Kemampuan ICL ini sangat krusial karena memungkinkan model beradaptasi dengan tugas baru melalui instruksi teks tanpa memerlukan pelatihan ulang (*retraining*) yang mahal.

Pada tahap inferensi, *LLM* menghasilkan token secara *autoregresif* berdasarkan distribusi probabilitas bersyarat:

$$y_t = \arg \max_y P_\theta(y | y_{<t}, x), \quad (2.4)$$

di mana y_t merupakan token yang dihasilkan pada langkah waktu t , yang ditentukan berdasarkan probabilitas bersyarat P_θ terhadap urutan token sebelumnya (y_1 hingga y_{t-1}) dan input x yang diberikan.

Namun, terlepas dari kecanggihannya, *LLM* memiliki keterbatasan fundamental yang

menjadi tantangan besar dalam domain akademik. Model ini bersifat probabilistik, bukan mesin penalaran logis murni, yang berarti model menghasilkan teks berdasarkan tebakan statistik terhadap kelanjutan kata yang paling mungkin, bukan berdasarkan pemahaman fakta yang absolut. Studi oleh ? menyebutkan bahwa hal ini menyebabkan munculnya halusinasi, di mana model memberikan jawaban yang terdengar meyakinkan namun salah secara faktual atau sepenuhnya fabrikasi. Selain itu, *LLM* mengalami kendala *knowledge cutoff*, di mana pengetahuan model terbatas pada data pelatihan terakhirnya, sehingga tidak mampu mengakses informasi terkini atau literatur ilmiah terbaru yang diterbitkan setelah masa pelatihannya.

Dalam penelitian ini, *LLM* berperan sebagai “otak” atau unit pemrosesan utama dalam arsitektur sistem yang bertanggung jawab untuk menghasilkan jawaban. Oleh karena itu, mengintegrasikan *LLM* dengan pengetahuan eksternal menjadi strategi penting untuk mengurangi risiko halusinasi dengan cara memberikan dasar (*grounding*) pada setiap respons generatif menggunakan fakta terstruktur yang dapat diverifikasi dan memiliki jejak data (*provenance*) yang jelas.

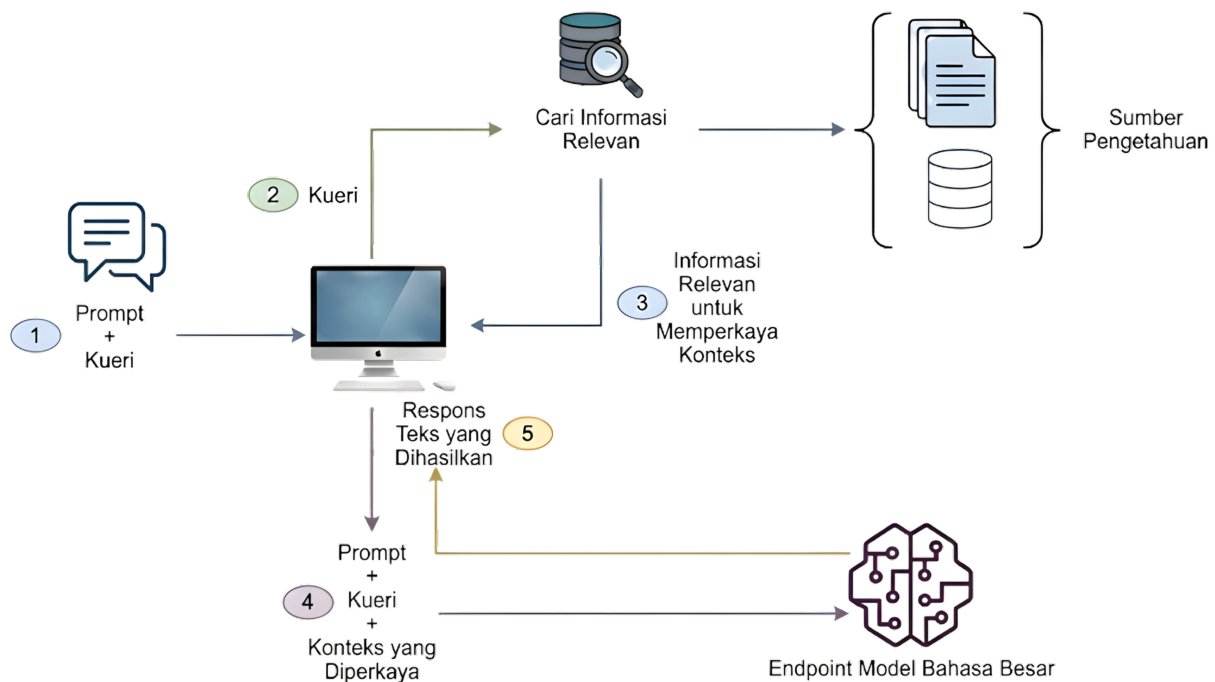
5. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) adalah sebuah arsitektur yang dirancang untuk meningkatkan performa model bahasa besar (*LLM*) dengan menggabungkan mekanisme pencarian informasi eksternal ke dalam proses generasi teks (?). Secara prinsip, *RAG* berperan sebagai penghubung antara kemampuan penalaran linguistik statis dari *LLM* dengan basis pengetahuan eksternal dinamis yang dapat diperbarui terus-menerus. Pendekatan ini menghasilkan konsep yang dikenal sebagai *grounded intelligence* atau kecerdasan yang berlandaskan bukti, di mana model tidak hanya mengandalkan pengetahuan internalnya, tetapi juga melakukan pencarian pada dokumen referensi sebelum memberikan respons (?).

Sebagaimana ditekankan oleh ?, pendekatan ini dirancang secara efektif untuk mitigasi keterbatasan inheren pada *LLM*, seperti fenomena halusinasi (*hallucination*), keterbatasan pengetahuan akibat masa pelatihan yang statis (*knowledge cutoff*), serta kurangnya spesialisasi pada domain tertentu. Dengan menyandarkan respons pada dokumen pendukung yang relevan, *RAG* memungkinkan model untuk menghasilkan jawaban yang lebih akurat dan terverifikasi. Secara umum, alur kerja standar *RAG* terdiri dari tiga tahapan utama, yaitu:

- A. **Indexing:** Segmentasi teks menjadi *chunks* dan konversi ke representasi vektor (*embedding*) dalam basis data.
- B. **Retrieval:** Pencarian kemiripan semantik (*semantic similarity*) untuk menemukan potongan teks relevan berdasarkan kueri pengguna.
- C. **Generation:** Sintesis jawaban oleh *LLM* menggunakan *augmented prompt* yang menggabungkan kueri asli dan konteks yang ditemukan.

Evolusi *RAG* telah melahirkan berbagai paradigma. ? mengklasifikasikan evolusi metode *RAG* ke dalam tiga paradigma utama. Pertama, *Naive RAG* yang menggunakan pola

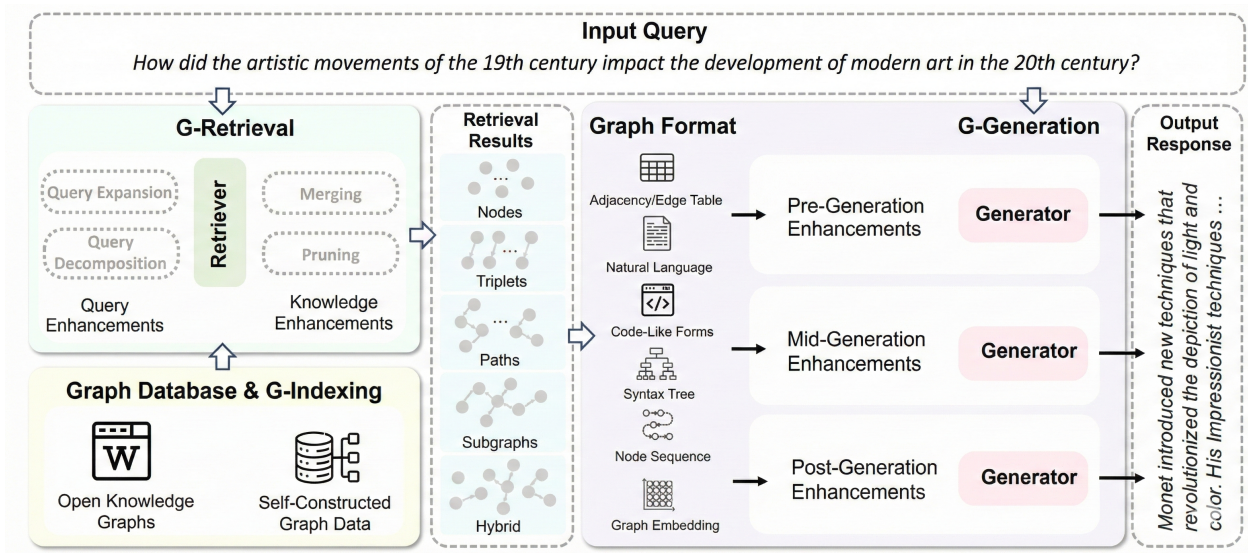


Gambar 2.3: Diagram Alur *Retrieval-Augmented Generation* (RAG)

retrieve-read sederhana melalui kesamaan vektor. Kedua, **Advanced RAG** yang menerapkan optimasi pra-pengambilan (seperti ekspansi kueri) dan pasca-pengambilan (seperti pemerincian ulang atau *reranking*) untuk meningkatkan presisi hasil (?). Selanjutnya, metode **Modular RAG** menawarkan fleksibilitas melalui integrasi modul fungsional untuk pengolahan data terstruktur seperti grafik atau tabel (?). Secara garis besar, paradigma-paradigma metode tersebut diilustrasikan pada ??.

Berbagai penelitian telah dilakukan untuk mengembangkan sistem RAG. Penelitian oleh ? melakukan studi pengembangan metode *Information Retrieval* pada RAG dan menemukan bahwa metode tersebut seringkali tidak dapat menemukan informasi relevan apabila pertanyaan bersifat kompleks. Penelitian terkini juga menunjukkan bahwa penggunaan satu metode pengambilan informasi kurang efektif. Mulai berkembang penelitian yang menggabungkan dua pendekatan yaitu pencarian berbasis kata kunci (*sparse retrieval*) dan pencarian berdasarkan makna atau konteks (*dense retrieval*). Penggabungan metode ini memungkinkan sistem mengambil informasi yang relevan meskipun penggunaan istilah berbeda tetapi memiliki makna serupa.

Namun, implementasi RAG konvensional yang hanya berbasis pada kemiripan vektor (*Vector RAG*) sering menghadapi tantangan berupa fragmentasi konteks, di mana hubungan antar-dokumen yang kompleks atau struktur hierarkis dalam literatur akademik cenderung terabaikan. Keterbatasan dalam melakukan penalaran pencarian pada data yang saling terkait inilah yang mendorong perlunya integrasi struktur data yang lebih kaya dan dapat membaca hubungan antar-dokumen yang kompleks.



Gambar 2.4: Alur Graph-Based RAG untuk tugas *Question-Answering* (?)

6. Graph-Based RAG

Graph-Based Retrieval-Augmented Generation (GraphRAG) merupakan evolusi dari paradigma RAG konvensional yang mengintegrasikan pengetahuan terstruktur dari *Knowledge Graph* ke dalam mekanisme *retrieval* model bahasa besar (?). Berbeda dengan *Vector RAG* tradisional yang hanya mengandalkan potongan teks (*chunks*) terfragmentasi berdasarkan kemiripan vektor, *GraphRAG* secara khusus mengekstraksi hubungan eksplisit antar entitas untuk memperkaya konteks yang akan diberikan kepada model bahasa (?). Pendekatan ini secara mendasar mengubah cara mesin melakukan navigasi pengambilan informasi dari sekadar mencari “kata-kata yang mirip” menjadi proses penalaran relasional yang mampu menghubungkan poin-poin data yang tersebar di berbagai dokumen.

Tujuan *GraphRAG* adalah menemukan jawaban optimal (a^*) untuk pertanyaan (q) berbasis Graf Pengetahuan ($\mathcal{G}$), diformulasikan sebagai:

$$a^* = \arg \max_{a \in A} p(a|q, \mathcal{G}) \quad (2.5)$$

di mana A merepresentasikan himpunan seluruh kemungkinan respons yang dapat dihasilkan. Untuk mencapai tujuan tersebut, distribusi probabilitas target $p(a | q, \mathcal{G})$ dimodelkan melalui kolaborasi dua komponen utama, yaitu model *graph retriever* $p_\theta(G | q, \mathcal{G})$ dan model *answer generator* $p_\phi(a | q, G)$, di mana θ dan ϕ adalah parameter yang dapat dilatih.

$$p(a | q, \mathcal{G}) \approx p_\phi(a | q, G^*) p_\theta(G^* | q, \mathcal{G}) \quad (2.6)$$

Di sini, p_θ mewakili parameter model *retriever* yang mencari subgraf relevan, sementara p_ϕ adalah model *generator* yang menyusun jawaban berdasarkan subgraf tersebut.

Secara arsitektural, implementasi *GraphRAG* terbagi menjadi tiga tahapan utama, sebagaimana diilustrasikan pada ??:

A. Graph-Based Indexing (G-Indexing): Tahap pembangunan basis data graf terstruktur

yang berfungsi sebagai sumber pengetahuan. Proses ini mengorganisir entitas (*nodes*) dan relasi (*edges*) sedemikian rupa untuk mengoptimalkan pencarian. Data graf ini dapat berasal dari sumber terbuka seperti Wikidata (?) atau dibangun secara mandiri dari kumpulan dokumen akademik tertentu. Indeksasi adalah tahap paling krusial karena menentukan seberapa detail informasi yang dapat diambil nantinya.

B. Graph-Guided Retrieval (G-Retrieval): Pada tahap pengambilan, sistem bertujuan mengambil elemen graf yang paling relevan, seperti pasangan subjek-predikat-objek (*triples*), jalur hubungan (*paths*), atau subgraf utuh, yang dapat diformulasikan sebagai:

$$\begin{aligned} G^* &= \mathbf{G-Retriever}(q, G) \\ &= \arg \max_{G \subseteq \mathcal{R}(G)} p_{\theta}(G|q, G) \\ &= \arg \max_{G \subseteq \mathcal{R}(G)} \mathbf{Sim}(q, G), \end{aligned} \quad (2.7)$$

C. Graph-Enhanced Generation (G-Generation): Pada tahap generasi, data graf yang diambil diintegrasikan ke dalam proses pembuatan jawaban. Struktur graf ini dapat diubah menjadi teks naratif atau diproses langsung oleh model *transformer* yang peka terhadap struktur graf agar menghasilkan respons yang koheren, informatif, dan valid, yang dapat dinotasikan sebagai:

$$\begin{aligned} a^* &= \mathbf{G-Generator}(q, G^*) \\ &= \arg \max_{a \in \mathcal{A}} p_{\phi}(a|q, G^*) \\ &= \arg \max_{a \in \mathcal{A}} p_{\phi}(a|F(q, G^*)), \end{aligned} \quad (2.8)$$

Salah satu inovasi penting dalam domain ini adalah kerangka kerja *Microsoft GraphRAG*, yang memperkenalkan konsep deteksi komunitas hierarkis dengan menggunakan algoritma *Leiden* (?). Metode ini membagi graf menjadi kluster-kluster yang kemudian diringkas menjadi laporan komunitas (*community reports*), sehingga memungkinkan sistem untuk menjawab kueri bersifat “global” (tema besar dalam satu dataset) yang biasanya gagal ditangani oleh RAG berbasis vektor. Namun, pendekatan ini menghadapi tantangan seperti tingginya biaya komputasi dan risiko cakupan komunitas yang terlalu luas saat skala data bertambah besar. Sebagai perbaikan, pendekatan *LightRAG* memitigasi hal ini dengan mekanisme pengambilan dua tingkat menggunakan kata kunci *lokal* dan *global* (?). Teknik ini secara signifikan mengurangi beban komputasi serta dipercaya dapat mempercepat pencarian. Namun, penerapan metode ini masih jarang dilakukan di lingkungan akademik yang berisiko mengalami hilangnya hubungan antar entitas jika tidak dipetakan secara eksplisit.

Merespons keterbatasan masing-masing pendekatan tersebut, penelitian ini menerapkan strategi *Hybrid Vector-Graph Retrieval* untuk mengoptimalkan kinerja sistem. Pendekatan hibrida ini diperlukan untuk menyinergikan keunggulan metode pengambilan informasi konvensional berbasis *semantic retrieval* (vektor) yang cepat dalam pencarian kemiripan,

dengan *structured retrieval* (graf) yang unggul dalam penalaran relasional. Sinergi ini dirancang untuk menyeimbangkan antara efisiensi komputasi dan akurasi kontekstual, sehingga sistem mampu menyajikan referensi ilmiah yang relevan dan mendalam tanpa menimbulkan *bottleneck* dalam proses *inference*.

(Halaman ini sengaja dikosongkan)

BAB III METODE PENELITIAN

A. Dataset

Penelitian ini menggunakan dataset primer yang disusun secara mandiri untuk merepresentasikan entitas dan aktivitas akademik pada domain Informatika dan Komputer (Infokom) di Universitas Negeri Surabaya (UNESA). Pengembangan dataset ini dilakukan untuk mengatasi keterbatasan ketersediaan basis data publik yang terintegrasi secara komprehensif. Perancangan atribut data difokuskan untuk mengoptimalkan mekanisme *Question-Answering* berbasis graf (*GraphRAG*), yang mensyaratkan pelacakan asal-usul informasi (*provenance tracking*) hingga ke dokumen asli guna memitigasi halusinasi LLM ?.

Dataset dihimpun melalui integrasi data dari tiga sumber utama: (1) profil akademik dosen pada pangkalan data PDDIKTI (?), (2) riwayat publikasi *Google Scholar* (?), dan (3) metadata artikel ilmiah dari *Scopus* (?). Data yang terkumpul mencakup informasi dari dosen di berbagai program studi di bawah rumpun Infokom UNESA, di antaranya: S1 Sains Data (13 dosen), S1 Kecerdasan Artifisial (5), S1 Teknik Informatika (9), S1 Sistem Informasi (10), S2 Informatika (6), dan S1 Teknik Elektro (24). Selain itu, dataset ini juga mencakup data dari program studi terkait seperti S1 Bisnis Digital dan kependidikan teknik lainnya untuk memperkaya cakupan *knowledge base*. Rincian struktur atribut disajikan pada Tabel ??.

Tabel 3.1: Struktur Atribut Dataset Penelitian

Dataset	Nama Atribut	Tipe Data	Deskripsi
Data Profil Dosen	nama_dosen	String	Nama lengkap dosen beserta gelar terverifikasi.
	nama_norm	String	Nama dosen yang dinormalisasi tanpa gelar.
	nip	String	Nomor Induk Pegawai.
	nidn	String	Nomor Induk Dosen Nasional.
	prodi	String	Program studi penempatan.
	scholar_id	String	ID unik profil Google Scholar dosen.
	scopus_id	String	ID unik profil Scopus dosen.
	sinta_id	String	ID unik profil SINTA dosen.

Bersambung ke halaman berikutnya

?? (lanjutan).

Dataset	Nama Atribut	Tipe Data	Deskripsi
	source	String	Sumber akuisisi data (mis. WEB+PDDIKTI, WEB).

Bersambung ke halaman berikutnya

?? (lanjutan).

Dataset	Nama Atribut	Tipe Data	Deskripsi
Data Publikasi Ilmiah <i>(Scopus & Scholar)</i>	Authors	String	Daftar nama penulis artikel ilmiah.
	Author IDs	String	Kumpulan ID profil penulis dari paper.
	Title	String	Judul artikel ilmiah publikasi.
	Year	Integer	Tahun publikasi untuk kebutuhan pencarian temporal.
	Journal	String	Nama jurnal, konferensi, atau prosiding publikasi.
	Link	String	Tautan langsung menuju halaman <i>publisher</i> atau PDF.
	Abstract	Text	Ringkasan isi artikel untuk rujukan utama RAG.
	Keywords	String	Kata kunci asli dari penulis untuk perkuatan <i>graph</i> .
	Document Type	String	Jenis publikasi (mis. <i>Article, Conference</i>).
	DOI	String	<i>Digital Object Identifier</i> sebagai resolusi identitas.
	TLDR	Text	<i>Too Long; Didn't Read</i> , intisari singkat oleh AI.

B. Alur Penelitian

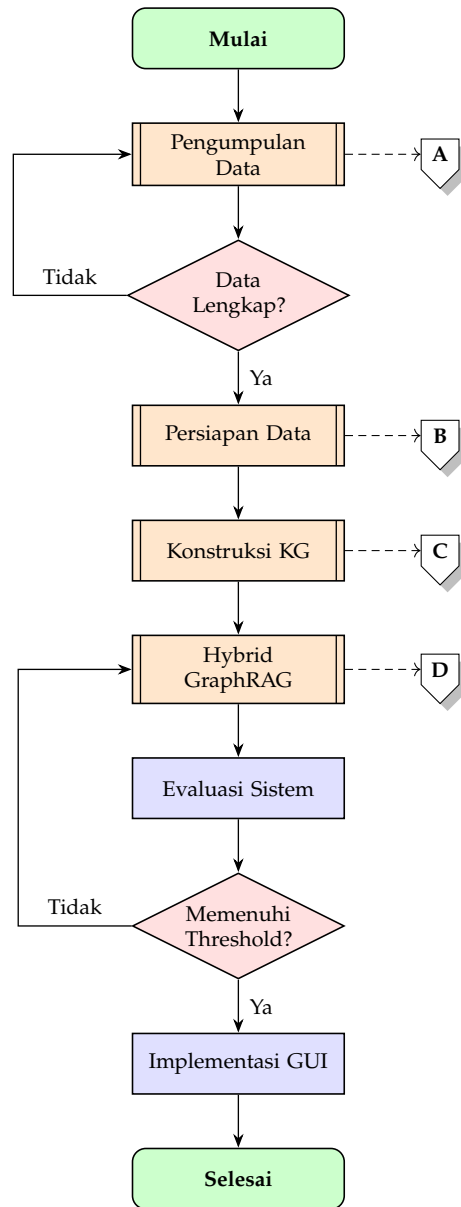
Alur penelitian dirancang sebagai rangkaian proses sistematis untuk membangun *Knowledge Graph* dan mengimplementasikan *Hybrid Vector-Graph Retrieval* dalam pencarian literatur domain Informatika dan Komputer di UNESA. Gambaran umum alur ini ditunjukkan pada ??.

1. Pengumpulan Data (*Data Collection*)

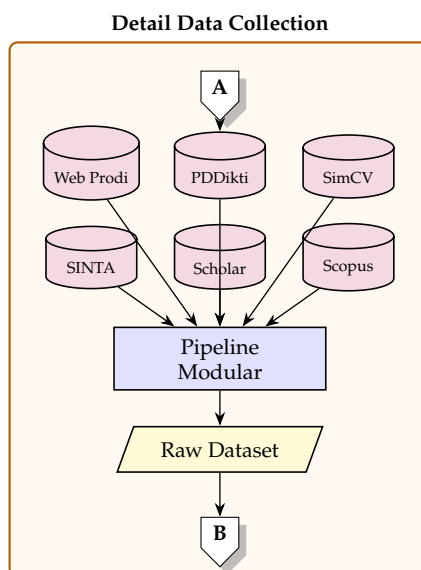
Tahap ini menghimpun dua kategori data utama: (1) data profil dosen beserta identitas indeksasinya, dan (2) data publikasi ilmiah beserta metadata pendukung. Kedua kategori ini menjadi fondasi untuk membangun *Scholarly Knowledge Graph (SKG)* yang merepresentasikan ekosistem publikasi Infokom UNESA. Proses akuisisi data dilakukan secara bertahap melalui *pipeline* modular yang mengintegrasikan enam sumber data, sebagaimana ditunjukkan pada ??.

a. Data Profil Dosen

Profil dosen dikumpulkan melalui *pipeline* bertahap yang menggabungkan beberapa sumber. Laman web resmi setiap program studi UNESA dijadikan sebagai sumber



Gambar 3.1: Alur metodologi penelitian



Gambar 3.2: Detail proses *Data Collection*

utama (*source of truth*) karena memuat nama lengkap beserta gelar, tautan profil akademik (*Google Scholar*, *Scopus*, *SINTA*), serta NIP dosen. Data ini kemudian diperkaya secara bertahap: API PDDikti (?) menyediakan NIDN dan NIP yang otoritatif, dilanjutkan dengan verifikasi identitas melalui *SimCV* UNESA (*cv.unesa.ac.id*) untuk memastikan keakuratan nama dan afiliasi program studi.

Selanjutnya, ID indeksasi dosen yang belum tersedia dari sumber sebelumnya dilengkapi melalui halaman departemen *SINTA* (?) untuk memperoleh *SINTA ID*, *SciVal* (?) untuk *Scopus ID*, serta verifikasi profil *Google Scholar* (?) untuk *Scholar ID*. Setiap tahap melakukan pencocokan nama (*fuzzy matching*) untuk mengasosiasikan data dari sumber berbeda ke entitas dosen yang sama, sehingga menghindari duplikasi.

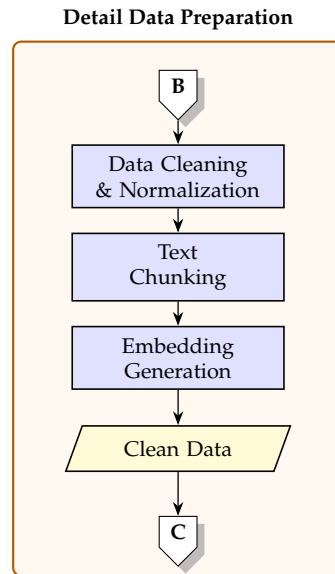
b. Data Publikasi Ilmiah

Data publikasi dihimpun dari dua sumber utama. Pertama, metadata publikasi terindeks *Scopus* diekspor secara otomatis melalui antarmuka *SciVal* (?) untuk setiap dosen yang telah memiliki *Scopus ID*. Data yang diperoleh meliputi judul, penulis, tahun, jurnal, DOI, dan tipe dokumen.

Kedua, publikasi dari *Google Scholar* (?) diakuisisi menggunakan *SerpAPI* berdasarkan *Scholar ID* masing-masing dosen. Sebelum disimpan, dilakukan deduplikasi silang terhadap data *Scopus* untuk mencegah data ganda. Seluruh publikasi kemudian diperkaya metadatanya: *CrossRef API* digunakan untuk memverifikasi DOI dan memperoleh kata kunci, *scraping* halaman *publisher* untuk mengekstrak abstrak, serta *Semantic Scholar API* untuk menghasilkan ringkasan singkat (*TLDR*). Strategi pengayaan bertingkat ini memaksimalkan kelengkapan data meskipun tidak semua sumber memberikan informasi yang sama.

2. Persiapan Data (*Data Preparation*)

Data mentah dari tahap sebelumnya selanjutnya diproses untuk menjamin kualitas *retrieval* (??), melalui tahapan berikut:



Gambar 3.3: Detail proses *Data Preparation*

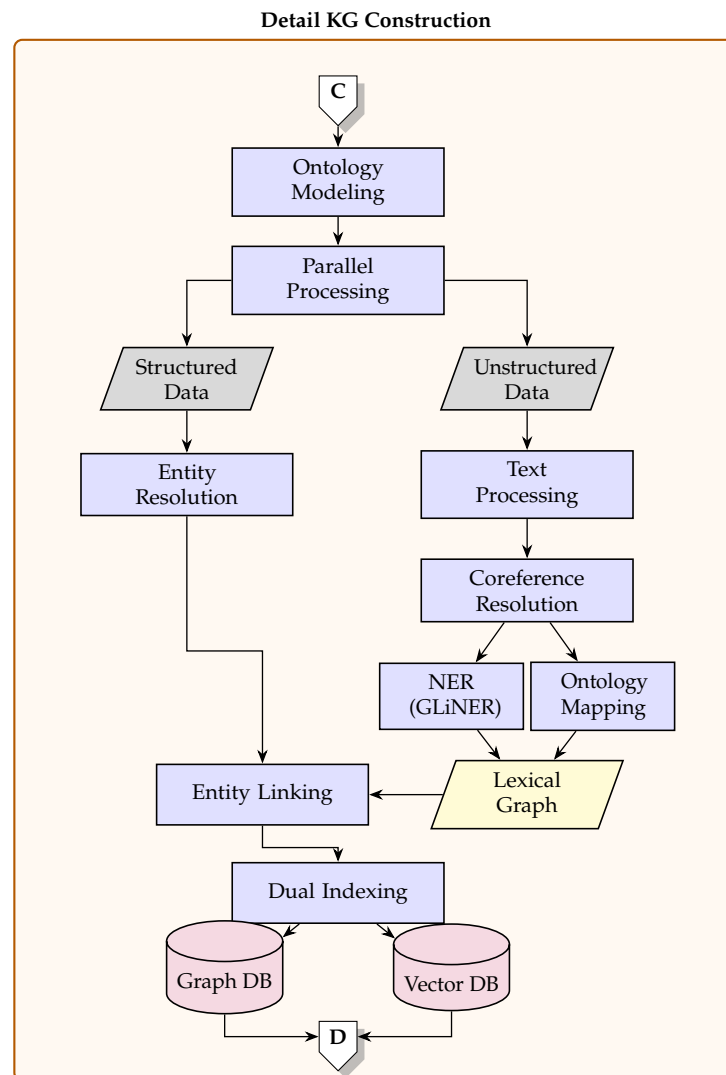
- a. ***Data Cleaning & Normalization***: Menangani *missing values* dan menstandarisasi variasi penulisan nama institusi serta singkatan gelar untuk mencegah duplikasi entitas dalam graf.
- b. ***Text Chunking (Pemotongan Teks)***: Judul, TLDR, abstrak, dan kata kunci digabungkan sebagai representasi tekstual publikasi, kemudian dipotong secara sadar kalimat dengan batas maksimum 2.024 karakter dan *overlap* 50 karakter. Strategi ini menjaga keutuhan unit informasi sekaligus membatasi *noise* dan penggunaan token.
- c. ***Embedding Generation***: Setiap *chunk*, entitas, relasi, dan himpunan kata kunci dikonversi menjadi vektor 1.024 dimensi menggunakan model Qwen/Qwen3-Embedding-0.6B melalui layanan SiliconFlow. Representasi tersebut memungkinkan pencarian berbasis makna (*semantic search*) yang melampaui pencocokan kata kunci leksikal.

3. Konstruksi *Knowledge Graph*

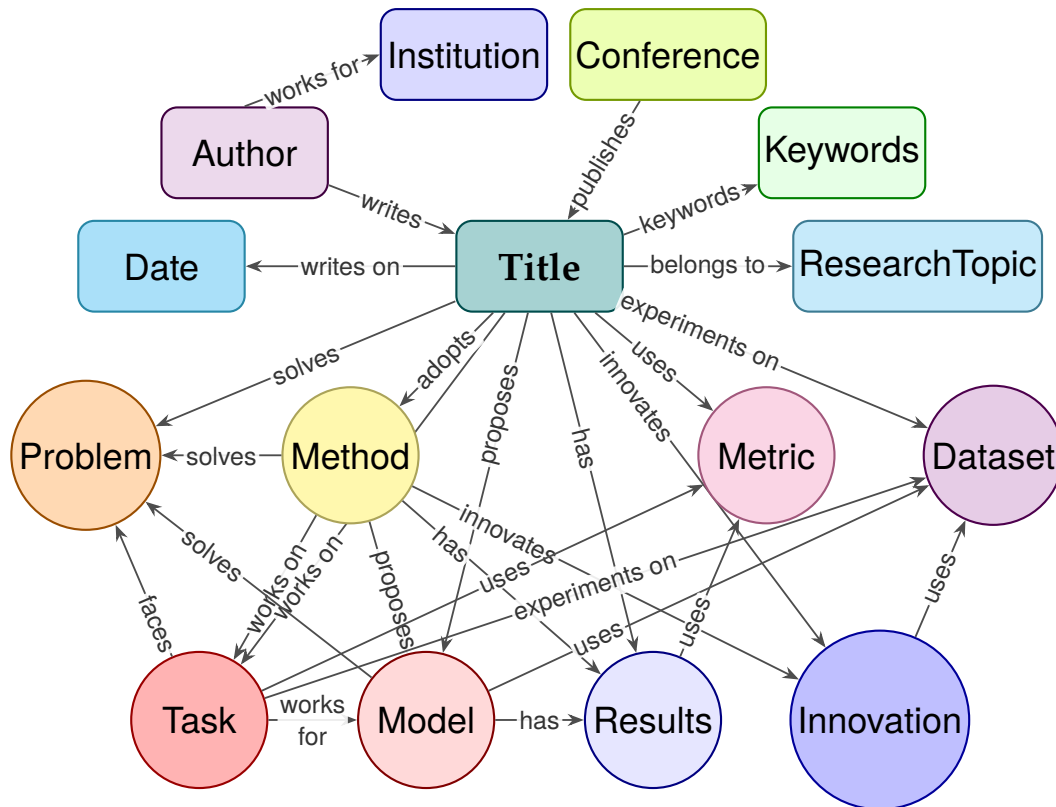
Tahap ini mentransformasi data yang telah dibersihkan menjadi basis pengetahuan terstruktur untuk penalaran relasional. Seperti ditunjukkan pada ??, data terstruktur (metadata dosen/publikasi) dan tidak terstruktur (abstrak) diproses melalui dua jalur paralel yang disatukan melalui *Entity Linking*.

1. *Ontology Modeling (Pemodelan Skema)*

Skema ontologi dirancang sebagai *blueprint Knowledge Graph* sebelum proses ekstraksi dimulai (??). Relasi aktif dikelompokkan menjadi: (1) relasi metadata akademik, seperti *publishes*, *has_author*, *has_affiliation*, *published_in_year*, *publi-*



Gambar 3.4: Detail tahapan konstruksi *Knowledge Graph*



Gambar 3.5: Skema ontologi (*Ontology Design*) *Knowledge Graph* untuk domain riset akademik

shed_in_venue, dan *collaborates_with*; (2) relasi semantik publikasi, seperti *has_topic*, *uses_method*, *uses_model*, *evaluated_with*, *has_result*, dan *belongs_to_domain*; serta (3) relasi SKOS untuk menghubungkan konsep IEEE. Relasi *cites* disediakan dalam rancangan, tetapi hanya diaktifkan apabila data referensi atau DOI yang disitasi tersedia. Pembatasan ini mencegah pembentukan relasi yang tidak didukung data sumber.

2. *Parallel Processing Pipeline*

Data akademik diproses melalui dua jalur paralel yang dioptimalkan untuk karakteristik data berbeda. Kedua jalur berjalan independen namun menghasilkan keluaran yang saling melengkapi (??).

a. Jalur Data Terstruktur: *Entity Resolution*

Data terstruktur (profil dosen dan afiliasi) diproses melalui *Entity Resolution* untuk menyatukan referensi duplikat. Variasi penulisan nama (misalnya “Aditya Prapanca” vs. “A. Prapanca”) diselesaikan untuk menghasilkan satu *canonical node* per dosen, menjamin akumulasi publikasi yang akurat pada satu entitas.

b. Jalur Data Tidak Terstruktur: *NLP-Based Extraction*

Tahap awal *Text Processing* membersihkan abstrak, TLDR, dan kata kunci dari karakter yang tidak relevan, lalu menyatukannya sebagai teks akademik untuk proses ekstraksi konsep.

Ekstraksi entitas tidak terstruktur diarahkan menggunakan *GLiNER* ? sebagai jalur opsional *zero-shot Named Entity Recognition*. Entitas yang ditemukan

tidak langsung dianggap sebagai relasi bebas. Setiap entitas terlebih dahulu dipetakan ke skema ontologi penelitian, seperti *Method*, *Model*, *Dataset*, *Metric*, *Domain*, atau *ResearchTopic*. Setelah tipe entitas jelas, relasi dibentuk secara deterministik mengikuti ontologi, misalnya *USES_MODEL*, *EVALUATED_WITH*, *USES_METHOD*, *HAS_TOPIC*, dan *BELONGS_TO_DOMAIN*. Dengan cara ini, graf tetap terkontrol dan tidak bergantung pada tebakan relasi bebas dari model. *GLiREL* diposisikan sebagai opsi eksperimen lanjutan, bukan jalur utama pada pembangunan graf yang dilaporkan.

3. Entity Linking

Tahap ini menjembatani kedua jalur dengan memetakan entitas hasil ekstraksi teks (nama dosen, topik riset) ke entitas kanonikal dari jalur terstruktur. Misalnya, penyebutan “Deep Learning” dalam abstrak dihubungkan ke node topik yang sudah ada, bukan membentuk node duplikat. Hasilnya adalah *Scholarly Knowledge Graph* utuh yang mengintegrasikan entitas terverifikasi dengan relasi dari teks tanpa duplikasi.

4. Dual Indexing

Untuk mendukung *Hybrid Retrieval*, hasil konstruksi KG disimpan dalam dua format berikut:

a. Vector Indexing

Representasi tekstual publikasi dipotong menjadi segmen maksimum 2.024 karakter dengan *overlap* 50 karakter. Setiap potongan teks, entitas, relasi, dan petunjuk kata kunci dikonversi menjadi vektor 1.024 dimensi menggunakan Qwen/Qwen3-Embedding-0.6B melalui SiliconFlow. Vektor disimpan pada Zilliz Cloud yang kompatibel dengan Milvus dalam koleksi PaperChunk, EntityEmbedding, RelationshipEmbedding, dan ContentKeyword.

b. Graph Indexing

Triplet entitas dan relasi dimuat ke basis data graf *Neo4j* melalui *Cypher queries*, memfasilitasi pencarian berbasis struktur relasional dan *graph traversal*.

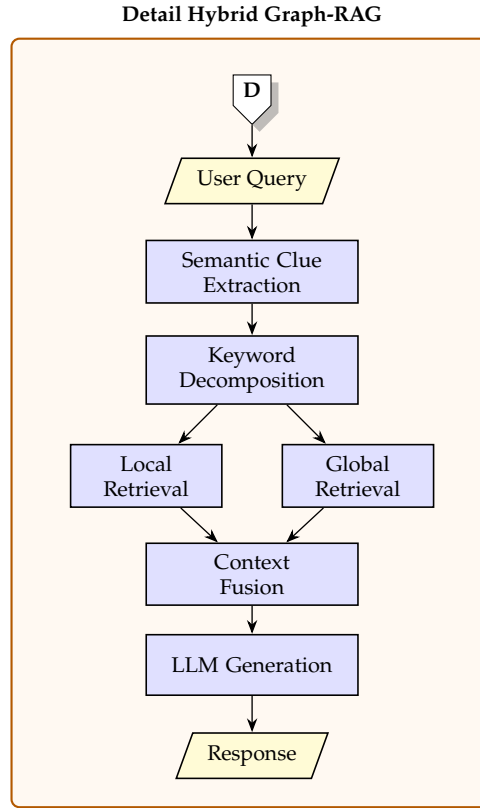
4. Hybrid Retrieval pada GraphRAG Model

Berbeda dengan RAG konvensional yang hanya mengandalkan pencarian vektor, pendekatan ini memperkuat *retrieval* dengan struktur *Knowledge Graph* (?). Kueri pengguna diproses melalui serangkaian tahap optimasi sebelum pengambilan data.

a. Pengambilan Petunjuk Semantik (*Clues*) pada Vector Database

Tahap ini mengidentifikasi entitas kunci dari pertanyaan pengguna via pencarian vektor, lalu memetakannya ke terminologi ontologi graf (misalnya “AI” dipetakan ke “Kecerdasan Buatan”).

Pertanyaan pengguna *Q* dipetakan ke ruang vektor menggunakan fungsi *embedding*, kemudian diukur kedekatannya dengan *keyword* korpus via *cosine similarity*. Seleksi kandidat *clues* didefinisikan sebagai:



Gambar 3.6: Detail proses *Hybrid Retrieval*

$$K^{\text{clues}} = \{k \in K^{\text{content}} \mid \text{sim}(f_{\text{embed}}(Q), f_{\text{embed}}(k)) \geq \tau\} \quad (3.1)$$

Setelah *clues* diekstraksi, model bahasa besar (LLM) mengubahnya menjadi dua tingkat kata kunci yang saling melengkapi:

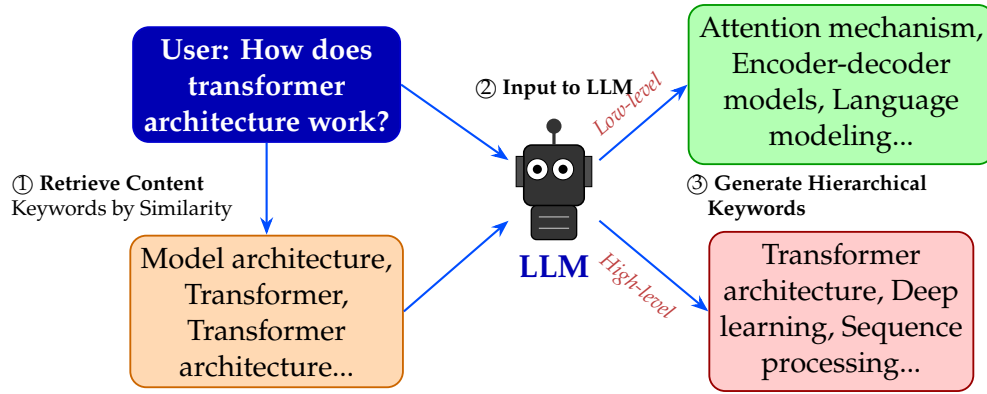
$$(\mathcal{K}_h, \mathcal{K}_l) = \text{LLM}_{\text{keyword}}(Q, K^{\text{clues}}) \quad (3.2)$$

Kata kunci *High-level* $\mathcal{K}_h$ menangkap konsep abstrak (misalnya “Deep Learning”, “Sequence Processing”), sementara *Low-level* $\mathcal{K}_l$ menangkap detail teknis spesifik (misalnya “Attention Mechanism”, “Encoder-Decoder Models”). Strategi dua tingkat ini memungkinkan sistem menjawab pertanyaan holistik ($\mathcal{K}_h$) maupun presisi tinggi ($\mathcal{K}_l$).

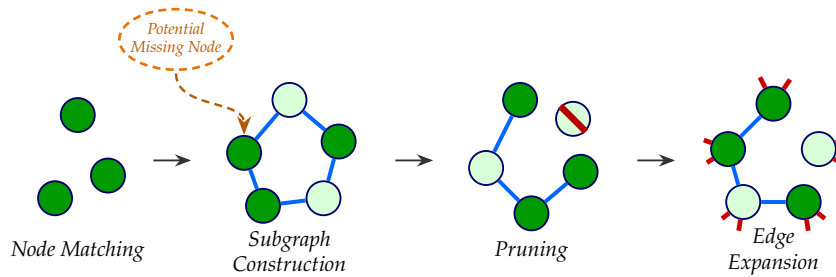
?? mengilustrasikan proses ini. LLM berfungsi sebagai *reasoning engine* yang mensintesis pertanyaan asli dengan petunjuk semantik, memecah kompleksitas kueri menjadi komponen pencarian yang relevan secara leksikal dan kontekstual. Pendekatan hierarkis ini meningkatkan *recall* sekaligus menjaga *precision*.

b. Ekstraksi Informasi Lokal Berbasis Subgraf (*Local Retrieval*)

Tahap ini menjawab pertanyaan yang membutuhkan ketepatan tinggi terhadap entitas tertentu. Sistem melakukan *anchor node matching* untuk menemukan entitas relevan, lalu menelusuri graf sejauh 1–2 hop untuk mengambil konteks tetangga. Metode ekstraksi subgraf diarahkan oleh kata kunci *low-level* $\mathcal{K}_l$ dengan tahapan



Gambar 3.7: Ilustrasi Ekstraksi Kata Kunci Berdasarkan Petunjuk (*clues*) (?)



Gambar 3.8: Tahapan Pembentukan Subgraf

berikut:

1. **Node Matching (Pencocokan Entitas):** *Graph nodes* dicocokkan dengan kata kunci *low level* ($\mathcal{K}_l$) menggunakan metrik *vector similarity* yang memprioritaskan *nodes* dengan nilai *cosine similarity* tertinggi terhadap *keyword embeddings*.
2. **Subgraph Construction:** Simpul-simpul jangkar dihubungkan menggunakan *shortest path algorithms* untuk menemukan rute paling efisien antar entitas terkait.
3. **Pruning:** *Node* atau *edge* dengan relevansi rendah dieliminasi agar subgraf tetap padat dan bermakna.
4. **Text Unit Extraction:** *Textual chunks* yang terasosiasi dengan *retained edges* dalam lingkup *one-hop ego-graph* diperingkat berdasarkan frekuensi kemunculannya. Segmen teratas diprioritaskan untuk *retrieval*.
5. **Edge Expansion:** Subgraf diperluas dengan menggabungkan *one-hop neighbors* dari *matched nodes* beserta *edges* yang memenuhi kriteria relevansi.
6. **Data Truncation:** Data yang diekstraksi (*nodes, edges, text units*) dipangkas hingga total token memenuhi ambang batas komputasi.

Proses ini menghasilkan konteks lokal terstruktur (??). Hasil akhir dinotasikan sebagai tiga komponen: entitas ($\mathcal{E}$), relasi ($\mathcal{R}$), dan unit teks ($\mathcal{T}$):

$$\mathcal{L}_{\text{local}} = (\mathcal{E}_{\text{local}}, \mathcal{R}_{\text{local}}, \mathcal{T}_{\text{local}}) = \text{pipeline}_{\text{local}}(\mathcal{K}_l, G). \quad (3.3)$$

di mana $\mathcal{L}_{\text{local}}$ merupakan fungsi dari kata kunci spesifik dan struktur graf, yang

direduksi menjadi subset informasi paling signifikan untuk menjawab kueri.

c. Ekstraksi Jejaring Global (*Global Edge Network Retrieval*)

Jalur ini menjawab pertanyaan eksploratif yang membutuhkan pemahaman tematik luas (*sensemaking*). Sistem menggunakan *High-level Keywords* $\mathcal{K}_h$ untuk mengekstraksi *Global Edge Network* berbasis vektor, tanpa batasan jarak topologi antar *nodes*. Tahapannya meliputi:

1. **Edge Retrieval:** Basis data vektor dikueri untuk mengidentifikasi *edges* yang representasi vektornya berkorelasi dengan kata kunci tingkat tinggi, dengan memprioritaskan *edges* yang merangkum hubungan konseptual makro (seperti kerangka teori atau tren lintas domain).
2. **Nodes Retrieval:** *Nodes* yang terkait dengan *edges* terpilih diekstraksi, kemudian diurutkan berdasarkan derajat keterhubungannya (*degree*).
3. **Edge Data Processing:** Metadata *edges* diekstraksi, meliputi *degree* (jumlah koneksi *source/target node*) dan *weight* (kekuatan hubungan semantik) untuk mengkuantifikasi signifikansinya.
4. **Integration and Ranking:** *Edges* diperingkat menggunakan metrik komposit yang menggabungkan skor relevansi dan bobot koneksi, memprioritaskan tautan konseptual dominan.
5. **Text Unit Extraction:** *Text chunks* terasosiasi dengan *edges* diperingkat berdasarkan kemiripan dengan kata kunci *global* dan diambil sebagai konteks pendukung.
6. **Data Truncation:** Data dipangkas hingga total token memenuhi ambang batas komputasi.

Mekanisme ini menghasilkan konteks global yang kaya akan informasi hubungan, yang diformulasikan dalam persamaan:

$$\mathcal{L}_{\text{global}} = (\mathcal{E}_{\text{global}}, \mathcal{R}_{\text{global}}, \mathcal{T}_{\text{global}}) = \text{pipeline}_{\text{global}}(\mathcal{K}_h, G). \quad (3.4)$$

Sinergi antara presisi tinggi *Local Retrieval* dan cakupan luas *Global Retrieval* menjadi keunggulan utama arsitektur *GraphRAG* dalam menangani kompleksitas literatur ilmiah.

d. LLM Generation & Response

Tahap ini merupakan fase akhir, di mana informasi yang telah diekstraksi dari *local* dan *global* disintesis. Konteks yang terintegrasi ini kemudian disatukan dengan kueri asli pengguna untuk membentuk sebuah *augmented prompt*. *Augmented prompt* ini kemudian dikirimkan ke *LLM* untuk menghasilkan jawaban akhir.

5. Evaluasi Sistem

Penelitian ini mengevaluasi kinerja sistem menggunakan pendekatan multi-lapis untuk mengukur kualitas pencarian dokumen referensi (*retrieval quality*) dan kualitas sintesis jawaban (*generation quality*). Kerangka kerja evaluasi terdiri dari 94 pertanyaan

uji, yang terbagi menjadi 90 *ranked cases* (pertanyaan dengan jawaban acuan/*ground truth*) dan 4 *guardrail cases* (pertanyaan di luar cakupan untuk menguji penolakan sistem). Kumpulan pertanyaan tersebut dikelompokkan ke dalam tiga kategori utama: Kategori A (faktual, $n = 30$), Kategori B (relasional, $n = 30$), dan Kategori C (*multi-hop*/sintesis, $n = 30$).

Evaluasi dilakukan melalui tiga lapisan pengukuran yang saling melengkapi sebagai berikut:

1. **Evaluasi Akurasi Retrieval (Layer 1):** Mengukur kemampuan pencarian dalam menemukan dokumen ilmiah yang relevan menggunakan metrik *Mean Reciprocal Rank* (MRR) dan *Hit@K* (untuk $K = 1, 5, 10$) (?).

Mean Reciprocal Rank (MRR) didefinisikan sebagai rata-rata kebalikan dari posisi peringkat dokumen relevan pertama yang berhasil ditemukan:

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i} \quad (3.5)$$

di mana $|Q|$ melambangkan total kueri dan rank_i merupakan peringkat posisi dokumen relevan pertama pada kueri ke- i .

Hit@K mengukur proporsi kueri di mana setidaknya satu dokumen relevan berhasil diambil dalam peringkat K teratas:

$$\text{Hit@K} = \frac{\text{Jumlah kueri dengan dokumen relevan di top } K}{|Q|} \quad (3.6)$$

2. **Evaluasi Kualitas Jawaban Berbasis RAGAS (Layer 2):** Mengevaluasi aspek kualitas pembuatan teks secara otomatis menggunakan kerangka kerja RAGAS (?). Metrik yang digunakan meliputi *Faithfulness* untuk mengukur tingkat kebebasan dari halusinasi dan *Answer Relevancy* untuk mengukur keselarasan respons terhadap pertanyaan.

Faithfulness didefinisikan sebagai rasio klaim atau pernyataan dalam jawaban (V) yang dapat diverifikasi secara langsung terhadap dokumen konteks pendukung (S):

$$\text{Faithfulness} = \frac{|V|}{|S|} \quad (3.7)$$

Answer Relevancy mengukur relevansi semantik jawaban yang dihasilkan terhadap kueri pengguna dengan menghitung rata-rata kemiripan kosinus (*cosine similarity*) dari representasi vektor kueri asli terhadap beberapa pertanyaan buatan yang dibandingkan dari jawaban tersebut:

$$\text{Answer Relevancy} = \frac{1}{m} \sum_{i=1}^m \cos(E_{q_i}, E_q) \quad (3.8)$$

di mana E_q adalah representasi vektor dari kueri asli, E_{q_i} adalah vektor kueri buatan ke- i , dan m adalah jumlah kueri buatan.

3. **Evaluasi Hakim LLM Berpasangan (Pairwise LLM Judge - Layer 4):** Menggunakan model bahasa besar (*DeepSeek v4-flash*) sebagai hakim otomatis (*LLM-as-a-Judge*) untuk membandingkan performa jawaban sistem yang diusulkan (*Subgraph* dan *Hybrid*) terhadap baseline (*Naive*) secara berpasangan (*pairwise*) (?). Evaluasi ini menilai respons berdasarkan empat dimensi utama: *Faithfulness*, *Traceability* (kemudahan penelusuran bukti referensi), *Comprehensiveness* (kelengkapan informasi), serta penilaian keseluruhan (*Overall*). Hasil evaluasi dinyatakan dalam bentuk *Win Rate* (persentase kemenangan terhadap baseline):

$$\text{Win Rate} = \frac{\text{Jumlah kueri dimenangkan oleh mode } M}{|Q|} \times 100\% \quad (3.9)$$

Melalui skenario evaluasi multi-lapis ini, kelebihan dan kelemahan masing-masing mode pencarian (*Naive*, *Subgraph*, dan *Hybrid*) dapat diidentifikasi secara komprehensif tanpa bergantung pada ambang batas keberhasilan tunggal. Hasil evaluasi disimpan secara terstruktur dan diekspor dalam bentuk tabel serta diagram radar yang disajikan pada Bab IV.

Mode naive diperlakukan sebagai representasi *Vector RAG* karena hanya mengandalkan pencarian semantik pada *PaperChunk*. Mode *subgraph* memanfaatkan simpul dan jalur graf lokal. Mode *global* menggunakan relasi yang diretrieval dari indeks *RelationshipEmbedding*. Mode *hybrid* menggabungkan konteks lokal dan global, sedangkan mode *mix* menggabungkan bukti tekstual, petunjuk kata kunci, entitas, relasi, dan subgraf. Hasil evaluasi disimpan dalam *notebooks/build-graph/outputs/evaluation* dan diekspor menjadi tabel serta gambar yang dapat langsung digunakan pada Bab IV.

6. Implementasi GUI (*GUI Implementation*)

Antarmuka pengguna dikembangkan sebagai aplikasi web yang mengadaptasi backbone Yuxi. Pada sisi pengguna, sistem disajikan sebagai antarmuka percakapan sehingga dosen atau pengguna lain dapat mengajukan pertanyaan dalam bahasa natural. Pada sisi belakang, pertanyaan tersebut diteruskan ke *agent* dan alat *query_kb* untuk menjalankan proses *Academic GraphRAG* terhadap Neo4j dan Zilliz/Milvus.

GUI tidak digunakan untuk membangun graf, tetapi menjadi lapisan interaksi dan inspeksi hasil. Antarmuka ini menampilkan jawaban akhir, sumber publikasi, potongan dokumen, *graph triples*, serta panel bukti akademik yang membantu pengguna melihat dari mana jawaban diperoleh. Dengan demikian, GUI diposisikan sebagai penghubung antara pengguna dan sistem *Hybrid GraphRAG*, bukan sebagai pengganti pipeline konstruksi KG.

C. JADWAL PENELITIAN

Jadwal penelitian disusun dalam ??, mencakup seluruh tahapan dari persiapan hingga penyelesaian skripsi selama enam bulan, namun tetap fleksibel menyesuaikan perkembangan progres.

Tabel 3.2: Pelaksanaan Penelitian

Kegiatan	Januari	Februari	Maret	April	Mei	Juni
Studi Literatur						
Pengumpulan Data						
Persiapan Data						
Konstruksi KG						
Implementasi Hybrid RAG						
Evaluasi Sistem						
Penulisan Skripsi						

(Halaman ini sengaja dikosongkan)

(Halaman ini sengaja dikosongkan)

LAMPIRAN A
Data dan lain-lain